Travail noté sur les vues et l'indexation

Pascal Perron-Abran

October 19, 2026

Abstract

Ce travail a été préparé dans le cadre du cours INF6408 - Informatique de l'analyse multidimensionnelle. L'objectif est de développer la compréhension de l'indexation, principalement les index bitmap et la representation ORDPATH pour un fichier XML. Comme objectif individuel, nous avons aussi décidé de travailler nos aptitudes LaTeX et Github.

Table des matières

1 Question 1 **3**

2 Question 2 **3**

3 Question 3 **3**

1 Question 1

Pour la première colonne, nous devons donc créer un bitmap par mois, pour un total de 12 bitmaps. Dans la deuxième colonne, 16 bitmaps et, pour la troisième colonne, dix bitmaps.

Dans ce scénario, 38 bitmaps seront créés.

2 Question 2

La représentation ORDPATH, est une représentation hiérarchique des étiquettes XML. Lors de la création de cette représentation, seuls les entiers impairs seront utilisés afin de permettre l'insertions de nouveaux éléments lorsque nécessaire [3]

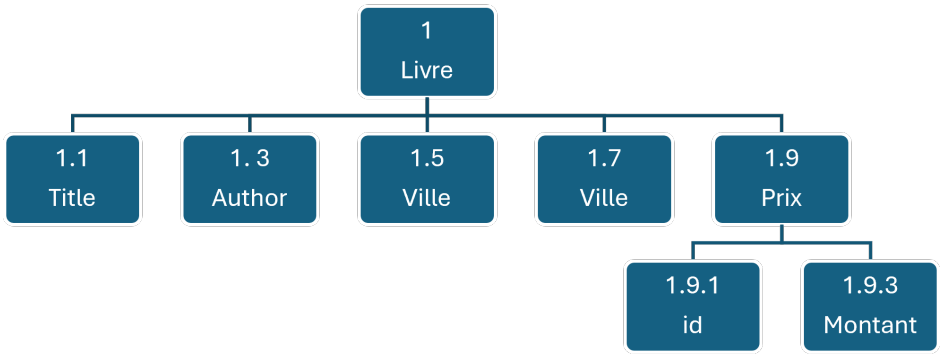


Figure 1: Représentation hiérarchique du fichier XML

ORDPATH	Nom	Type	Valeur
1	book	Élément	S.O.
1.1	title	Élément	La vie est belle
1.3	author	Élément	Jean Égard
1.5	ville	Élément	Montréal
1.7	ville	Élément	Paris
1.9	prix	Élément	S.O.
1.9.1	id	Attribut	14444
1.9.3	montant	Élément	104.00

Table 1: Representation ORDPATH d'un document XML

3 Question 3

Considérons que la base de données de la question 1 contienne n enregistrements, est organisée en $m = 3$ colonnes et que chaque valeur est stockée dans un registre de taille s. Si elle est organisée en *rangée*, et que nous voulons faire une recherche sur k colonne, tel que $k \leq 3$, nous pouvons utiliser l'équation 3.1 pour calculer la taille des données chargées [1].

$$D_{rangée} = n \cdot s \cdot m \tag{3.1}$$

Si nous avons la même base de données orientée *colonne* et que nous voulons faire la même requête sur k colonnes, la taille des données chargées peut etre obtenue par l'équation 3.2 (ibid.). Il est intéressant de noter

que, en combinant les équations 3.1 et 3.2, nous trouvons que le ratio entre la taille des données chargées ne dépend que du ratio k/m (voir l'équation 3.3).

$$D_{colonne} = n \cdot s \cdot k \tag{3.2}$$

$$\frac{D_{colonne}}{D_{rangée}} = \frac{n \cdot s \cdot k}{n \cdot s \cdot m} = \frac{k}{m} \tag{3.3}$$

Par conséquent, l'orientation *colonne* nous permet de charger moins de données que l'orientation *rangée*, tant que le nombre de colonne sur lesquelles nous faisons une requête est de $k \leq 2$ et, si $k = m$, les deux orientations auront la même performance.

De plus, nous pouvons optimiser les performances de l'orientation colonne en triant les données selon la colonne ayant le moins de valeurs possible (faible cardinalité). Dans notre cas, ce tri serait basé sur la colonne *Province*. Nous pourrions ensuite procéder au codage par plage. En reprenant l'exemple de la question d'approfondissement [2], et considérant que la colonne *Mois* a douze valeurs possibles, la colonne *Année*, seize, et la colonne *Province*, dix, notre pire scénario contiendrait 10 x 12 x 16 = 1 920 plages.

References

[1] Sarah Lee. *Column-oriented databases 101*. Oct. 2026. URL: https://www.numberanalytics.com/blog/column-oriented-databases-ultimate-guide.

[2] Daniel Lemire. *Indexation des données multidimensionnelles*. Sept. 2026. URL: https://lemire.me/notesdecours/entrepot/semaine5.html.

[3] Patrick O'Neil et al. "ORDPATHs: insert-friendly XML node labels". In: *Proceedings of the 2004 ACM SIGMOD International Conference on Management of Data*. SIGMOD '04. Paris, France: Association for Computing Machinery, 2004, pp. 903–908. ISBN: 1581138598. DOI: 10.1145/1007568.1007686. URL: https://doi.org/10.1145/1007568.1007686.